THE NEW COLLEGE

CHENNAI-14.

DEPARTMENT OF COMPUTER APPLICATIONS

OPEN SOURCE TECHNOLOGIES – LAB

PRACTICAL - VII

YA ALLAH
INCREASE MY KNOWLEDGE

III BCA – C

SEMESTER – VI

NAME : MUHAMMED EHATHISAM N

REG.NO : 2313181033227

April - 2026

THE NEW COLLEGE, CHENNAI-14.

DEPARTMENT OF COMPUTER APPLICATIONS (SHIFT-II)

NAME : Muhammed Ehathisam N

REGISTER NO : 2313181033227

CLASS : III - BCA SECTION: C

SUBJECT : OPEN SOURCE TECHNOLOGIES – PRACTICAL VII

SUBJECT CODE : 20BHMP607

Certified to be Bonafide Record of work done in the Computer Applications Laboratory of The New College, during the academic year **2023 – 2026** by the above candidate.

Lecturer-in-charge **Head of the Department**

Submitted for the Practical Examination held on 06-04-2026 in Computer Applications Laboratory of The New College.

Internal Examiner **External Examiner**

Place: Chennai

Date: 06-04-2026

INDEX

EX.NO	DATE	TITLE OF THE PROGRAM	PAGE NO	TEACHER'S SIGN
1	21/11/25	Install Node.js	01	
2	28/11/25	Create Web Servers Using Node.js	02	
3	09/12/25	Using NPM Command to Install Dependencies or Modules in the Application	04	
4	18/12/25	Use the Flask Framework to Route Web Pages and Retrieve Values from the Form	06	
5	05/01/26	Add Social Networks to your Web Project	09	
6	12/01/26	Create a Navigation Menu that Highlights the Selected Entry	12	
7	21/01/26	Create a Data Grid with Search Option	14	
8	28/01/26	Manipulate a Web Page Using React	16	
9	06/02/26	Attach Different Events to Elements of a Web Page	18	
10	17/02/26	Provide Different Effects Using React	21	
11	27/02/26	Install MySQL and Create Local and Remote Database	24	
12	05/03/26	Create a Full Stack Web Application – Smart Exam Cheating Detection System	29	

Ex No : 1 **Install Node.js**

Date : 21/11/25

Aim: To install Node.js on Windows 11 and verify its installation using command-line tools.

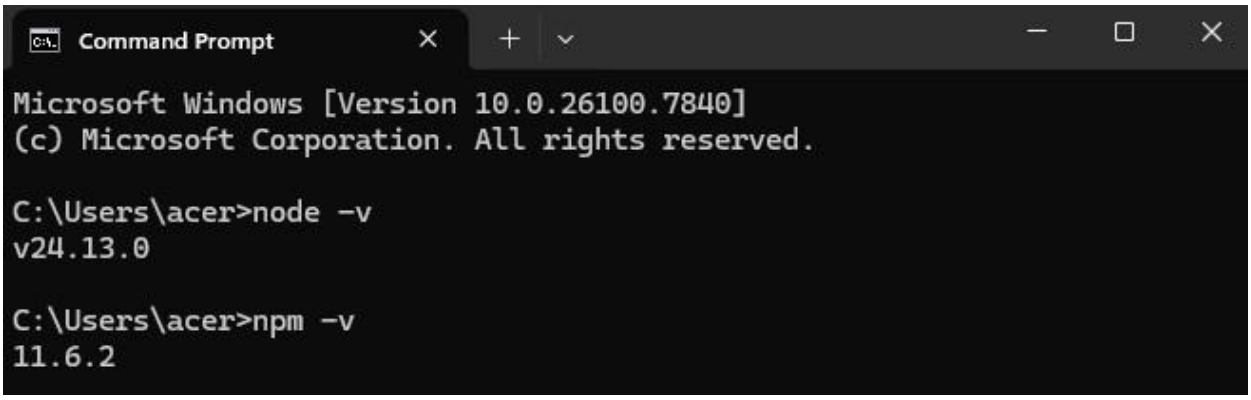
Procedure:

1. Open a web browser and go to https://nodejs.org.
2. Download the LTS (Long Term Support) version for Windows.
3. Run the downloaded .msi installer file.
4. Follow the setup wizard and keep the default installation settings.
5. Click Install and complete the installation process.
6. Open Command Prompt.
7. Type node -v and npm -v to verify the installation.

Source Code / Commands:

node -v
npm -v

Output:



Result: Thus, Node.js and NPM were successfully installed on Windows 11 and verified using the command prompt with node -v and npm -v.

Ex No : 2 **Create Web Servers Using Node.js**

Date : 28/11/25

Aim: To create a simple web server using Node.js and display output in a web browser.

Procedure:

1. Create a new folder named "webserver" on your Desktop.
2. Open the folder and create a new file named server.js.
3. Open server.js in a text editor (Notepad or VS Code).
4. Type the Node.js web server code and save the file.
5. Open Command Prompt and navigate to the folder using the cd command.
6. Run the server using the command node server.js.
7. Open a web browser and go to http://localhost:3000.

Source Code / Commands:

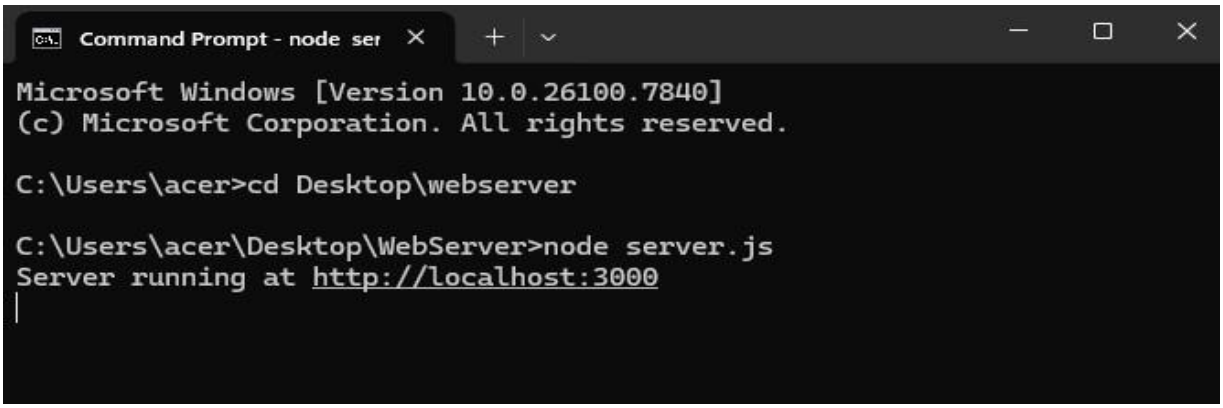
server.js:

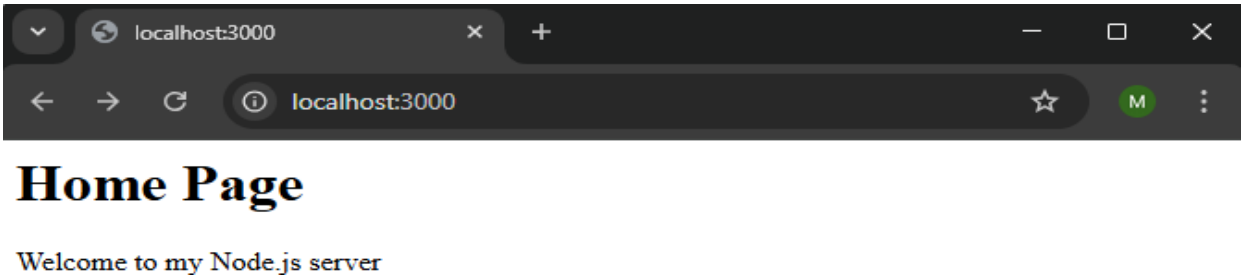
```
const http = require("http");
const server = http.createServer((req, res) => {
if (req.url === "/") {
res.writeHead(200, { "Content-Type": "text/html" });
res.end("<h1>Home Page</h1><p>Welcome to my Node.js server</p>");
}
else if (req.url === "/about") {
res.writeHead(200, { "Content-Type": "text/html" });
res.end("<h1>About Page</h1><p>This is the about page</p>");
}
else {
res.writeHead(404, { "Content-Type": "text/html" });
res.end("<h1>404 Page Not Found</h1>");
}
});
server.listen(3000, () => {
console.log("Server running at http://localhost:3000");
});
```

Command Prompt:

cd Desktop\webserver
node server.js

Output:





Result: Thus, a simple web server was successfully created using Node.js. The server ran on http://localhost:3000 and displayed different web pages such as Home, About, and 404 based on the URL.

Ex No : 3 Using NPM Command to Install Dependencies or Modules in the Application

Date : 09/12/25

Aim: To install a Node.js module using the NPM command for a web application.

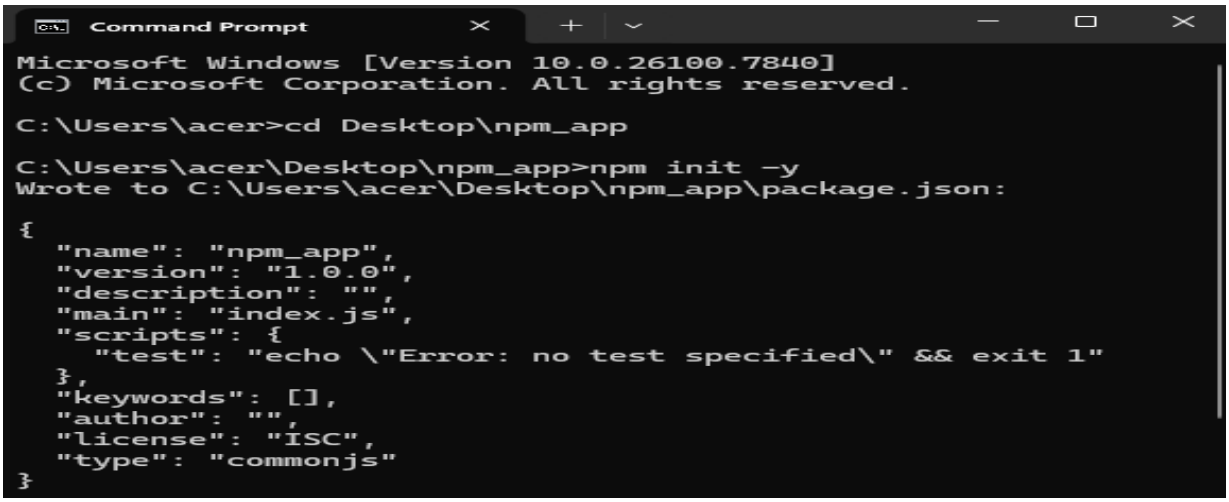
Procedure:

1. Create a new folder named npm_app on your Desktop.
2. Open Command Prompt and navigate to the folder using the cd Desktop\npm_app command.
3. Initialize the Node.js application using the command npm init -y.
4. Install a module (express) using the command `npm install express`.
5. Wait for the installation to complete.
6. Observe the created node_modules folder and package.json file.
7. Verify installation using the command npm list express.

Source Code / Commands:

cd Desktop\npm_app
npm init -y
npm install express
npm list express

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26100.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\acer>cd Desktop\npm_app

C:\Users\acer\Desktop\npm_app>npm init -y
Wrote to C:\Users\acer\Desktop\npm_app\package.json:

{
  "name": "npm_app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
Command Prompt

C:\Users\acer\Desktop\npm_app>npm install express

added 65 packages, and audited 66 packages in 4s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\acer\Desktop\npm_app>npm list express
npm_app@1.0.0 C:\Users\acer\Desktop\npm_app
`-- express@5.2.1
```

Result: Thus, the Node.js application was successfully initialized using NPM and the Express module was installed and verified through the npm list express command.

Ex No : 4 Use the Flask Framework to Route Web Pages and Retrieve Values from the Form

Date : 18/12/25

Aim: To use Flask framework for routing web pages and retrieving user inputs in the Smart Exam Cheating Detection System, and process the data using a rule-based algorithm to identify potential cheating activities.

Procedure:

1. Install required Python packages such as Flask, Flask-CORS, and JWT.
2. Create a Flask application file (app_improved.py) to initialize the backend server.
3. Enable CORS to allow communication between frontend and backend.
4. Create route modules such as auth_routes, exam_routes, and admin_routes.
5. Register these routes in the Flask application using Blueprints.
6. Define a login API route to receive user credentials from the frontend.
7. Retrieve the email and password from the request and validate the input.
8. Generate a JWT token and send the login response back to the frontend.

Source Code / Commands:

Backend Server Route Configuration (Backend/app_improved.py)

This mounts the authentication routes in the Flask application.

```
from flask import Flask
from flask_cors import CORS
from routes.auth_routes_improved import auth_bp
from routes.exam_routes import exam_bp
from routes.admin_routes import admin_bp
app = Flask(__name__)
# Enable CORS
CORS(app)
# Mount routes
app.register_blueprint(auth_bp, url_prefix="/api/auth")
app.register_blueprint(exam_bp, url_prefix="/api/exam")
app.register_blueprint(admin_bp, url_prefix="/api/admin")
if __name__== "__main__":
app.run(port=5000, debug=True)
```

Authentication Route Definition (Backend/routes/auth_routes_improved.py)
This defines the Flask route for candidate login.

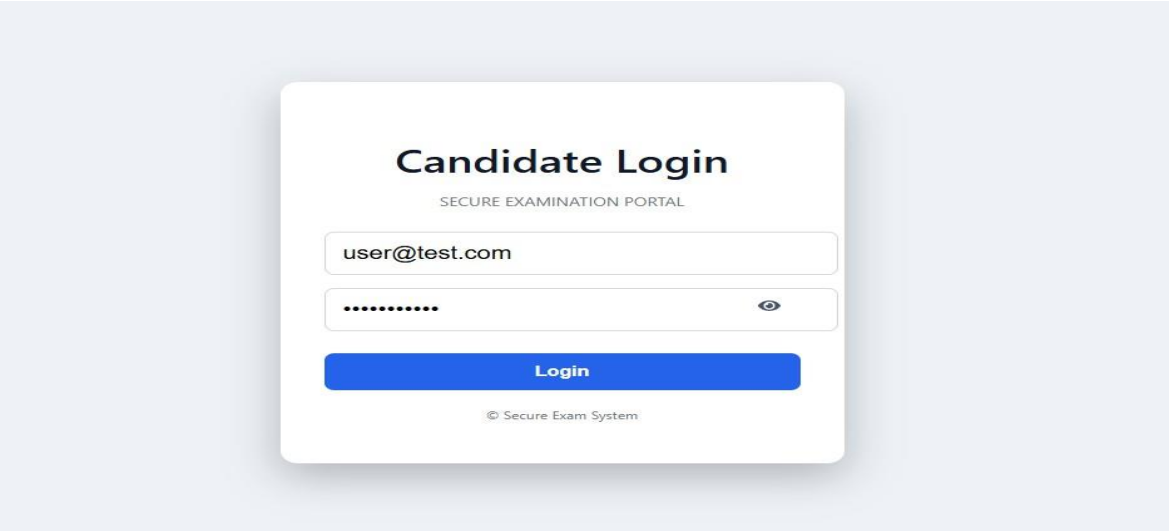
```
from flask import Blueprint, request, jsonify
from werkzeug.security import check_password_hash
import jwt
from datetime import datetime, timedelta
from config_improved import Config
from models.user_model import get_user_by_email
auth_bp = Blueprint("auth", __name__)
# Login Route
@auth_bp.route("/login", methods=["POST"])
def login():
data = request.json
email = data.get("email")
password = data.get("password")
# Validation
if not email or not password:
return jsonify({"error": "Email and password required"}), 400
# Get user from database
user = get_user_by_email(email)
if not user:
return jsonify({"error": "Invalid email or password"}), 401
# Check password
if not check_password_hash(user["password_hash"], password):
return jsonify({"error": "Invalid email or password"}), 401
# Generate token
token = jwt.encode({
"user_id": user["id"],
"email": user["email"],
"exp": datetime.utcnow() + timedelta(hours=Config.JWT_EXPIRATION_HOURS)
}, Config.JWT_SECRET, algorithm="HS256")
return jsonify({
"message": "Login successful",
"token": token
})
```

Frontend Login Request (Frontend/src/pages/Login_improved.jsx)
This sends login form data from the Smart Exam Cheating Detection System frontend to the Flask backend.

```
const handleLogin = async (e) => {
e.preventDefault();
setError("");
// Attempt login
setLoading(true);
try {
```

```
const result = await loginUser(email, password);
if (result.ok && result.data.token) {
// Store authentication token
localStorage.setItem("examToken",  result.data.token);
localStorage.setItem("user",  JSON.stringify(result.data.user));
// Redirect user
navigate("/mfa");
} else {
setError("Login failed");
}
} catch (err) {
console.error("Login error:", err);
} finally {
setLoading(false);
}
};
```

Output:



Result: Thus, the Flask framework was successfully used to create backend routes, handle user login requests, and retrieve form data from the frontend of the Smart Exam Cheating Detection System.

Ex No : 5 **Add Social Networks to Your Web Project**

Date : 05/01/26

Aim: To integrate social network links into the Smart Exam Cheating Detection System by adding GitHub and LinkedIn icons that redirect users to the developer's social media profiles.

Procedure:

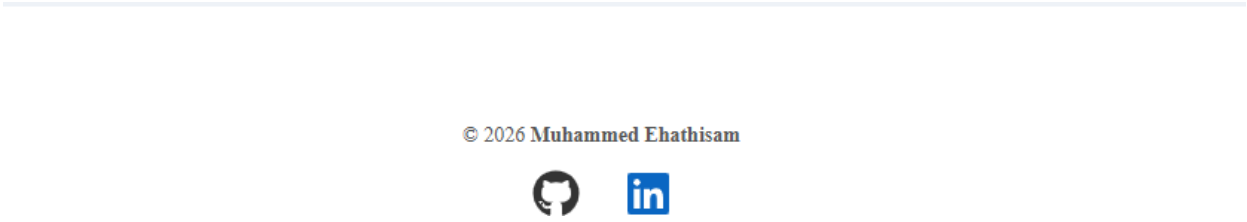
1. Open the frontend project in the code editor.
2. Install the React Icons library to use social media icons.
3. Create a Footer component in the components folder.
4. Import GitHub and LinkedIn icons from the React Icons library.
5. Define the developer's GitHub and LinkedIn profile URLs.
6. Add anchor tags to link the icons with the respective URLs.
7. Apply basic styling to align the icons and footer content properly.
8. Run the application and verify that the icons redirect to the social profiles.

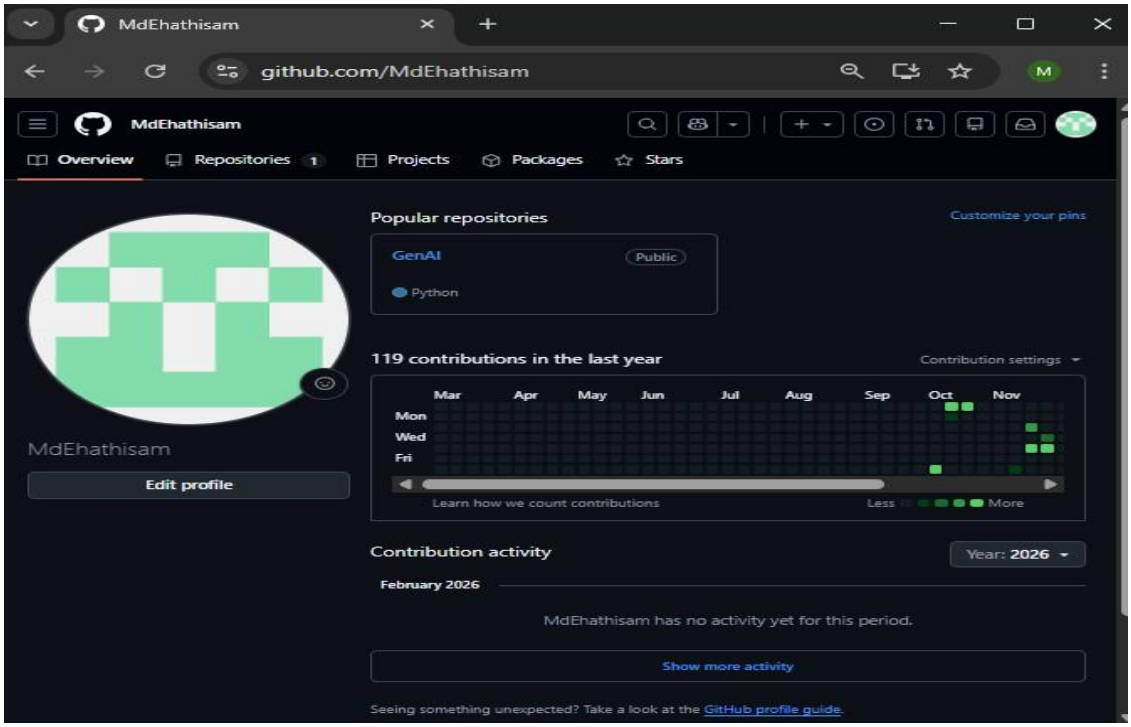
Source Code / Commands:
(Frontend/src/components/Login_improved.jsx)
This component displays GitHub and LinkedIn icons and redirects users to the corresponding social media profiles when clicked.

```
import React from "react";
import { FaGithub, FaLinkedin } from "react-icons/fa";
export default function Footer() {
const year = new Date().getFullYear();
const githubUrl = "https://github.com/MdEhathisam";
const linkedinUrl = "https://www.linkedin.com/in/muhammed-ehathisam-n- 80856b385";
return (
<footer
style={{
width: "100%",
padding: "25px 0",
textAlign: "center",
marginTop: "40px"
}}
>
<p style={{ fontSize: "14px", color: "#555" }}>
©{year} <strong>Muhammed Ehathisam</strong>
</p>
<div
```

```
style={{
marginTop: "15px",
display: "flex",
justifyContent: "center",
gap: "30px"
}}
>
{/* GitHub */}
<a
href={githubUrl}
target="_blank"
rel="noreferrer"
style={{ fontSize: "32px", color: "#333" }}
>
<FaGithub />
</a>
{/* LinkedIn */}
<a
href={linkedinUrl}
target="_blank"
rel="noreferrer"
style={{ fontSize: "32px", color: "#0A66C2" }}
>
<FaLinkedin />
</a>
</div>
</footer>
);
}
```

Output:





Result: Thus, social network links were successfully integrated into the web project. GitHub and LinkedIn icons were added to the footer and they redirect users to the developer's social profiles.

Ex No : 6 Create a Navigation Menu that Highlights the Selected Entry

Date : 12/01/26

Aim: To create a navigation menu in the Smart Exam Cheating Detection System that highlights the selected menu entry based on the active section in the Admin Dashboard.

Procedure:

1. Open the AdminDashboard component in the frontend project.
2. Create a navigation menu using the <nav> element.
3. Add buttons for sections such as Monitoring, Users, Config, Analytics, System, and Security.
4. Create a state variable called activeTab to track the selected menu item.
5. Attach onClick events to update the active tab.
6. Apply conditional class names to highlight the selected item.
7. Use CSS styles to visually indicate the active navigation entry.
8. Run the application and verify that the selected menu item is highlighted.

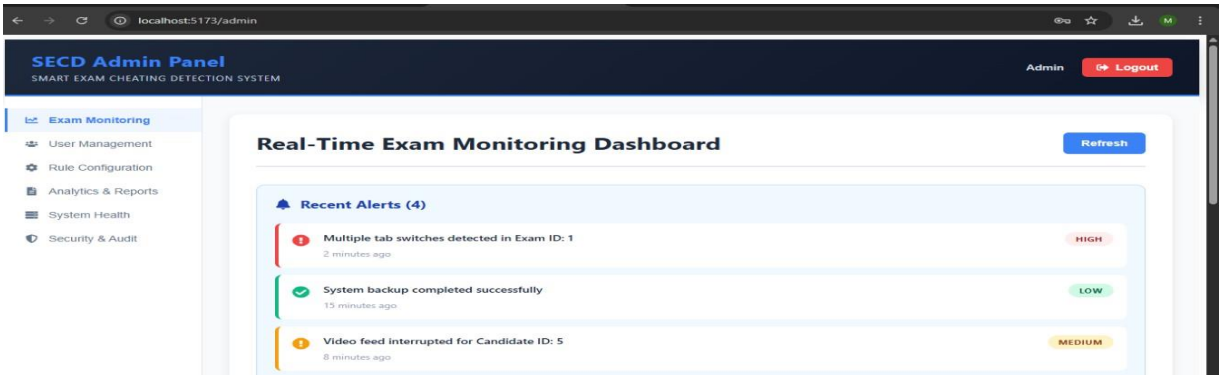
Source Code / Commands

Admin Dashboard Navigation
(Frontend/src/components/AdminDashboard.jsx)
This component creates a navigation menu and highlights the selected menu item using React state.

```
<nav className="admin-nav">
<button
className={`nav-item ${activeTab === "monitoring" ? "active" : ""}`}
onClick={() => setActiveTab("monitoring")}
>
<FaChartLine /> Exam Monitoring
</button>
<button
className={`nav-item ${activeTab === "users" ? "active" : ""}`}
onClick={() => setActiveTab("users")}
>
<FaUsers /> User Management
</button>
<button
className={`nav-item ${activeTab === "config" ? "active" : ""}`}
```

```
onClick={() => setActiveTab("config")}
>
<FaCog /> Rule Configuration
</button>
<button
className={`nav-item ${activeTab === "analytics" ? "active" : ""}`}
onClick={() => setActiveTab("analytics")}
>
<FaFileAlt /> Analytics & Reports
</button>
<button
className={`nav-item ${activeTab === "system" ? "active" : ""}`}
onClick={() => setActiveTab("system")}
>
<FaServer /> System Health
</button>
<button
className={`nav-item ${activeTab === "security" ? "active" : ""}`}
onClick={() => setActiveTab("security")}
>
<FaShieldAlt /> Security & Audit
</button>
</nav>
```

Output:



Result: Thus, a navigation menu was successfully created in the Admin Dashboard and the selected menu item was highlighted dynamically based on the active section.

Ex No : 7 **Create a Data Grid with Search Option**

Date : 21/01/26

Aim: To create a searchable data grid in the Smart Exam Cheating Detection System that allows the administrator to view and search exam sessions.

Procedure:

1. Open the AdminDashboard component in the frontend project.
2. Create state variables to store exam data and search input.
3. Add a search input field in the dashboard interface.
4. Capture the search text using the onChange event.
5. Convert the search input to lowercase for easier comparison.
6. Filter the exam list based on the entered search keyword.
7. Display the filtered exam list using the map function.
8. Run the application and verify that the grid updates based on the search.

Source Code / Commands:
(Frontend/src/ components /AdminDashboard.jsx)

1. State for Exams and Search

```
// State for exams and search
const [exams, setExams] = useState([]);
const [search, setSearch] = useState("");
```

2. Search Input Field

```
<input
  type="text"
  placeholder="Search Exam"
  value={search}
  onChange={(e) => setSearch(e.target.value)}
/>
```

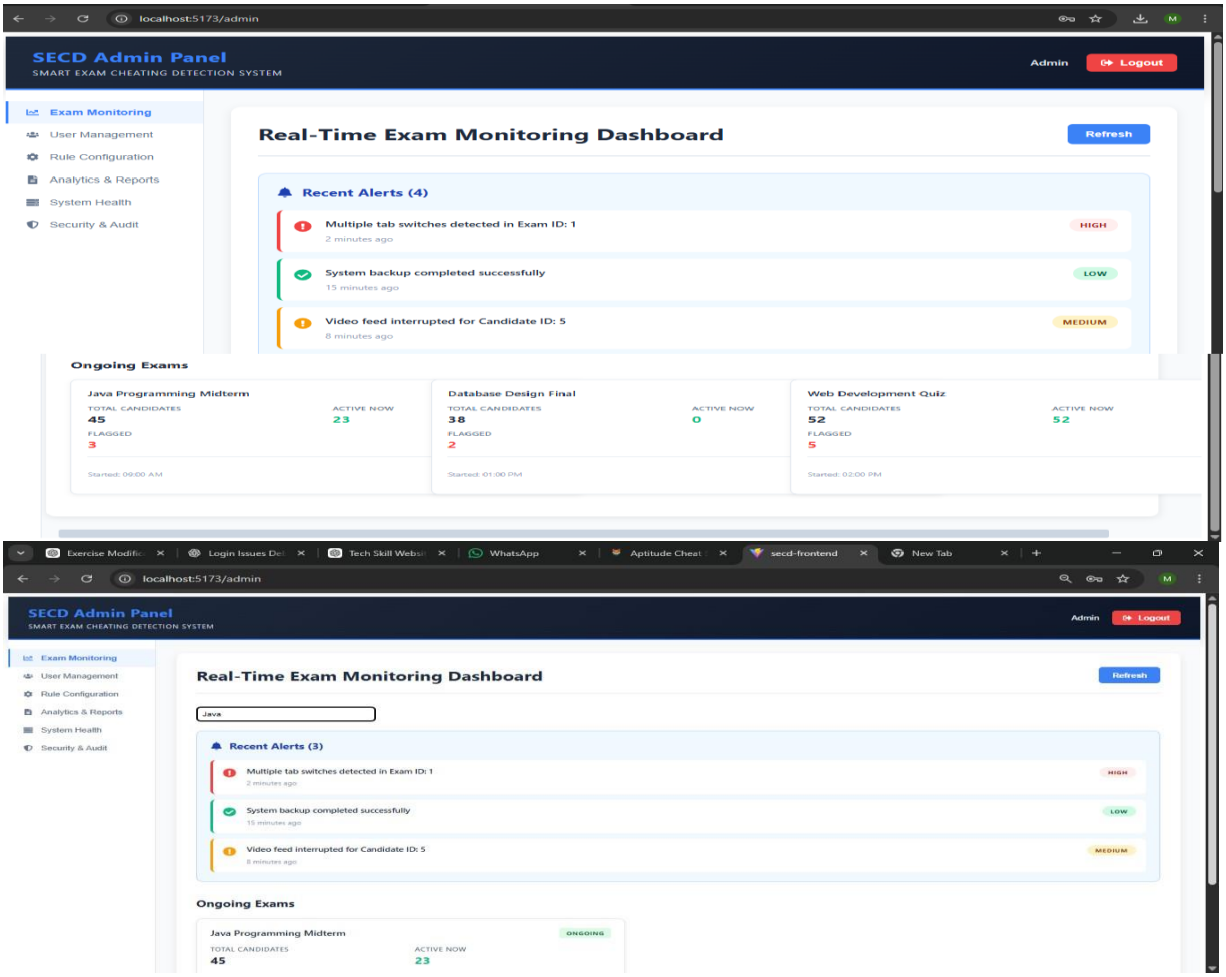
3. Filter Exams Based on Search

```
const filteredExams = exams.filter((exam) =>
exam.title.toLowerCase().includes(search.toLowerCase())
);
```

4. Display Exam Grid

```
1.  {filteredExams.map((exam) => (
2.    <div key={exam.id}>
3.      <h4>{exam.title}</h4>
4.      <p>Total Candidates: {exam.candidates}</p>
5.      <p>Active Now: {exam.active}</p>
6.      <p>Flagged: {exam.flagged}</p>
7.    </div>
8.  ))}
```

Output:



Result: Thus, a searchable data grid was successfully implemented allowing the administrator to search and view exam sessions dynamically.

Ex No : 8 **Manipulate a Web Page Using React**

Date : 28/01/26

Aim: To dynamically manipulate the Smart Exam Cheating Detection System web page by displaying exam details in a modal window when the administrator selects an exam.

Procedure:

1. Open the AdminDashboard component in the frontend project.
2. Create state variables for selected exam and modal visibility.
3. Define a function to open the exam details modal.
4. Attach the function to the View Details button.
5. Store the selected exam data when the button is clicked.
6. Display exam details dynamically inside the modal window.
7. Add a Close button to hide the modal window.
8. Run the application and verify that exam details appear dynamically.

Source Code / Commands

(Frontend/src components /AdminDashboard.jsx)

1. State Variables Used for Web Page Manipulation

```
1. const [selectedExam, setSelectedExam] = useState(null)
2. const [showExamModal, setShowExamModal] = useState(false)
```

2. Function to Open Exam Details Modal

```
1. const openExamModal = (exam) => {
2. setSelectedExam(exam)
3. setShowExamModal(true)
4. }
```

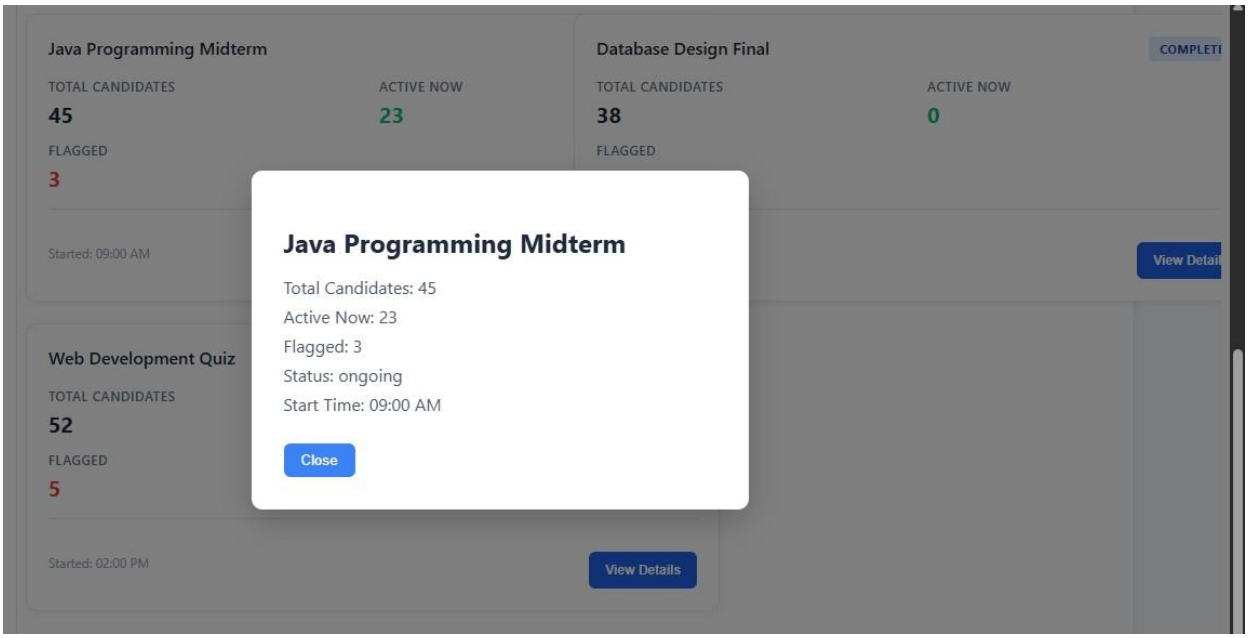
3. Button Click Event

```
1. <button
2. className="view-btn"
3. onClick={() => openExamModal(exam)}
4. >
5. View Details
6. </button>
```

4. Dynamic Modal Rendering

```
{showExamModal && selectedExam && (
<div className="modal-overlay">
<div className="modal-box">
<h2>{selectedExam.title}</h2>
<p>Total Candidates: {selectedExam.candidates}</p>
<p>Active Now: {selectedExam.active}</p>
<p>Flagged: {selectedExam.flagged}</p>
<p>Status: {selectedExam.status}</p>
<p>Start Time: {selectedExam.startTime}</p>
<button onClick={closeExamModal}>
    Close
</button>
</div>
</div>
)}
```

Output:



Result: Thus, the web page was successfully manipulated dynamically by displaying exam details inside a modal window when an exam was selected.

Ex No : 9 **Attach Different Events to Elements of a Web Page**

Date : 06/02/26

Aim: To attach different events to elements of the Smart Exam Cheating Detection System web page such as click, change, and submit events for managing exam sessions.

Procedure:

1. Open the AdminDashboard component in the frontend project.
2. Create buttons and input fields for user interactions.
3. Attach a click event to open exam details.
4. Attach a click event to refresh dashboard data.
5. Add a button to open the question upload modal.
6. Create a search input field to search exams.
7. Attach a change event to capture search input.
8. Attach a submit event to handle question bank uploads.

Source Code / Commands

(Frontend/src components /AdminDashboard.jsx)

1. Click Event – Open Exam Details

```
<button
  className="view-btn"
  onClick={() => openExamModal(exam)}
>
  View Details
</button>
```

2. Click Event – Refresh Dashboard Data

```
<button
  className="refresh-btn"
  onClick={handleRefresh}
 >
  Refresh
</button>
```

3. Click Event – Upload Question Bank Modal

```
<button
 className="action-btn"
 onClick={() => setShowUploadModal(true)}
>
 Manage Questions
</button>
```

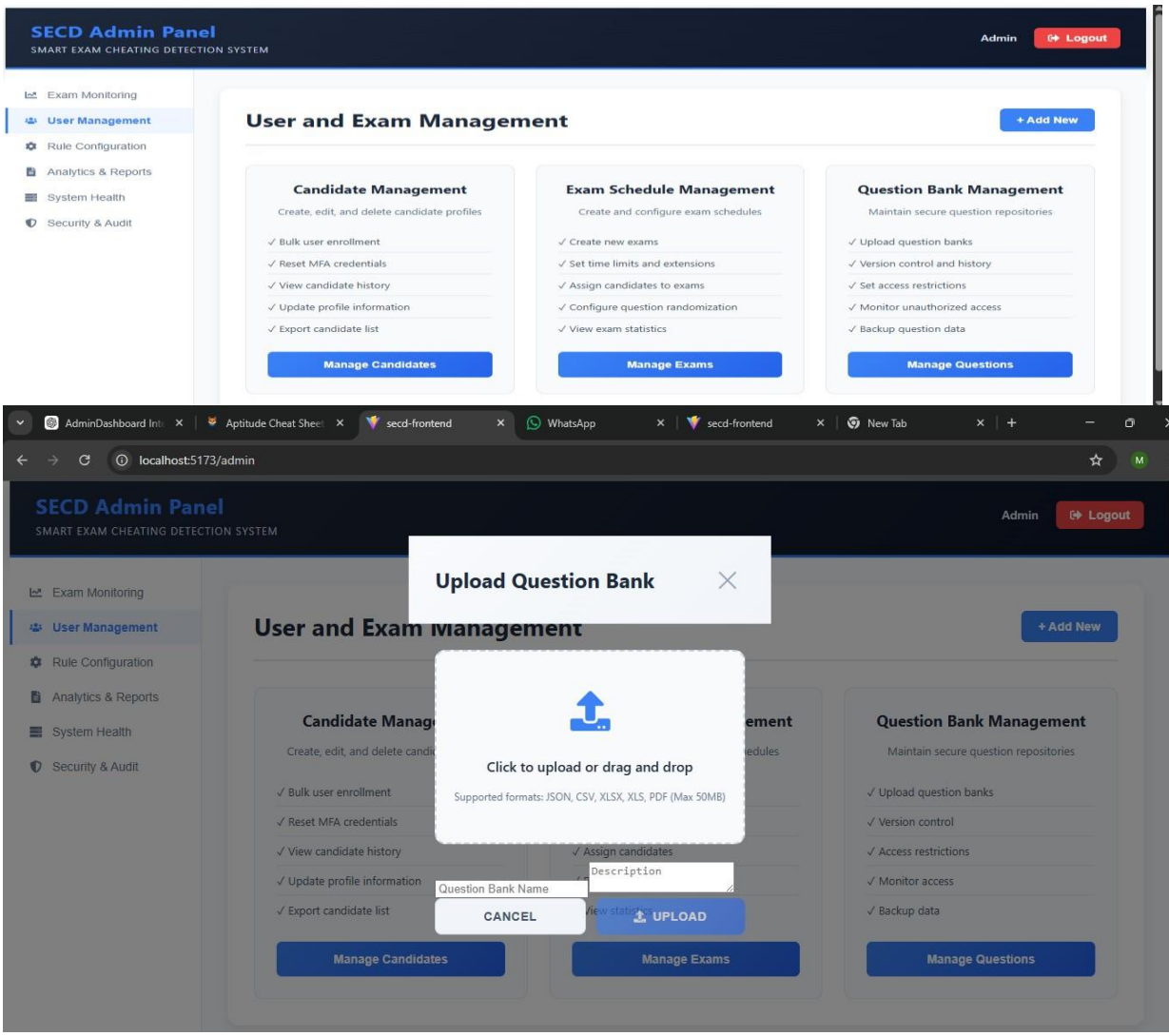
4. Change Event – Search Exam

```
<input
 type="text"
 placeholder="Search  Exam..."
 value={search}
 onChange={(e) => setSearch(e.target.value)}
 />
```

5. Submit Event – Upload Question Bank

```
<form onSubmit={handleUpload}>
<input
 type="text"
 value={fileName}
 onChange={(e) => setFileName(e.target.value)}
 placeholder="Question Bank Name"
/>
</form>
//Submit Handler:
const handleUpload = (e) => {
e.preventDefault();
if (!selectedFile) return;onSuccess({
name: fileName,
description,
fileName: selectedFile.name,
size: selectedFile.size
});
};
```

Output:



Result: Thus, various events such as click, change, and submit were successfully attached to web page elements to manage exam sessions and user actions.

Ex No : 10 **Provide Different Effects Using React**

Date : 17/02/26

Aim: To provide visual effects in the Smart Exam Cheating Detection System by dynamically showing and hiding elements such as loading indicators, exam alerts, uploaded question banks, and exam cards.

Procedure:

1. Open the AdminDashboard component in the frontend project.
2. Create buttons and input fields for user interactions.
3. Attach a click event to open exam details.
4. Attach a click event to refresh dashboard data.
5. Add a button to open the question upload modal.
6. Create a search input field to search exams.
7. Attach a change event to capture search input.
8. Attach a submit event to handle question bank uploads.

Source Code / Commands

(Frontend/src/ components /AdminDashboard.jsx)

1. Loading Effect

```
const handleRefresh = () => {
  setLoading(true);
  setTimeout(() => {
    loadDemoData();
    setLoading(false);
  }, 500);
};
<button
  className="refresh-btn"
  onClick={handleRefresh}
>
  {loading ? "Refreshing..." : "Refresh"}
</button>
```

2. Dynamic Alerts Display

```
{alerts.map((alert) => (
  <div
    key={alert.id}
    className={`alert-item alert-${alert.severity}`}
  >
  <div class="alert-content">
    <p>{alert.message}</p>
      <small>{alert.timestamp}</small>
  </div>
  </div>
))}
```

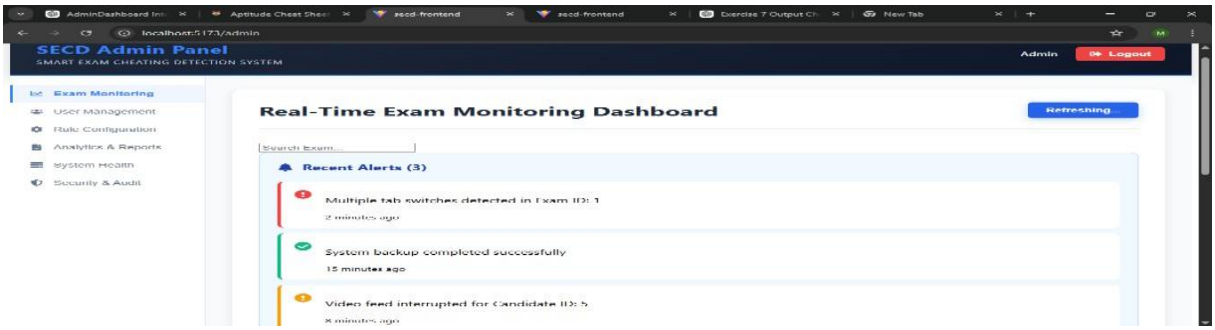
3. Conditional Exam Cards Display

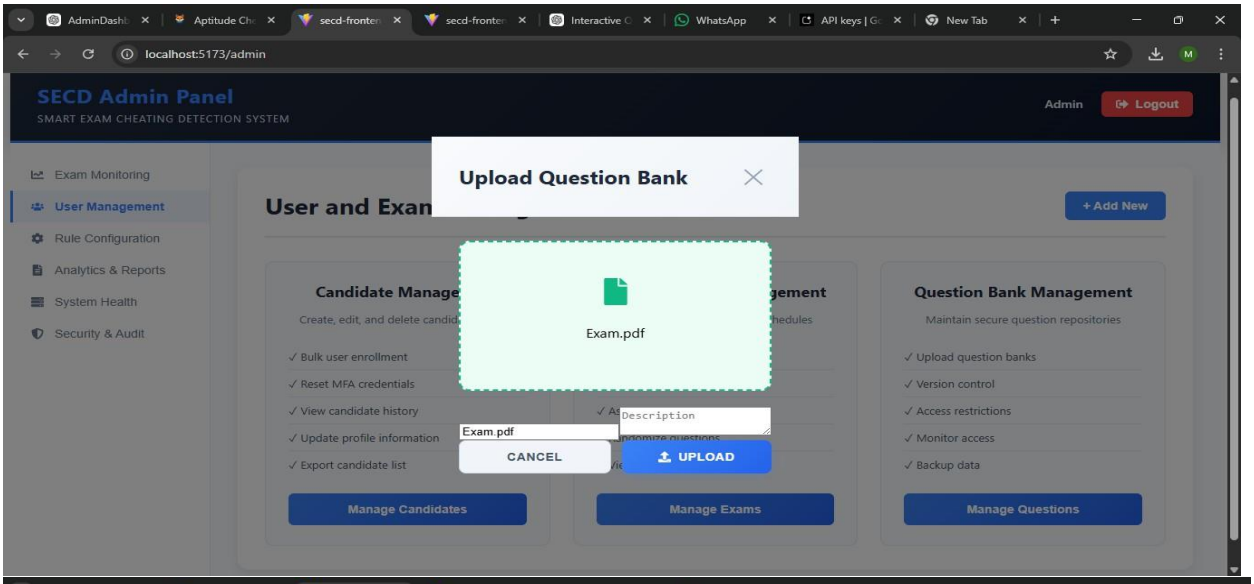
```
{filteredExams.map((exam) => (
  <div key={exam.id} class="exam-card">
    <h4>{exam.title}</h4>
     <span class="status-badge ${exam.status}`}>
     {exam.status.toUpperCase()}
    </span>
  </div>
))}
```

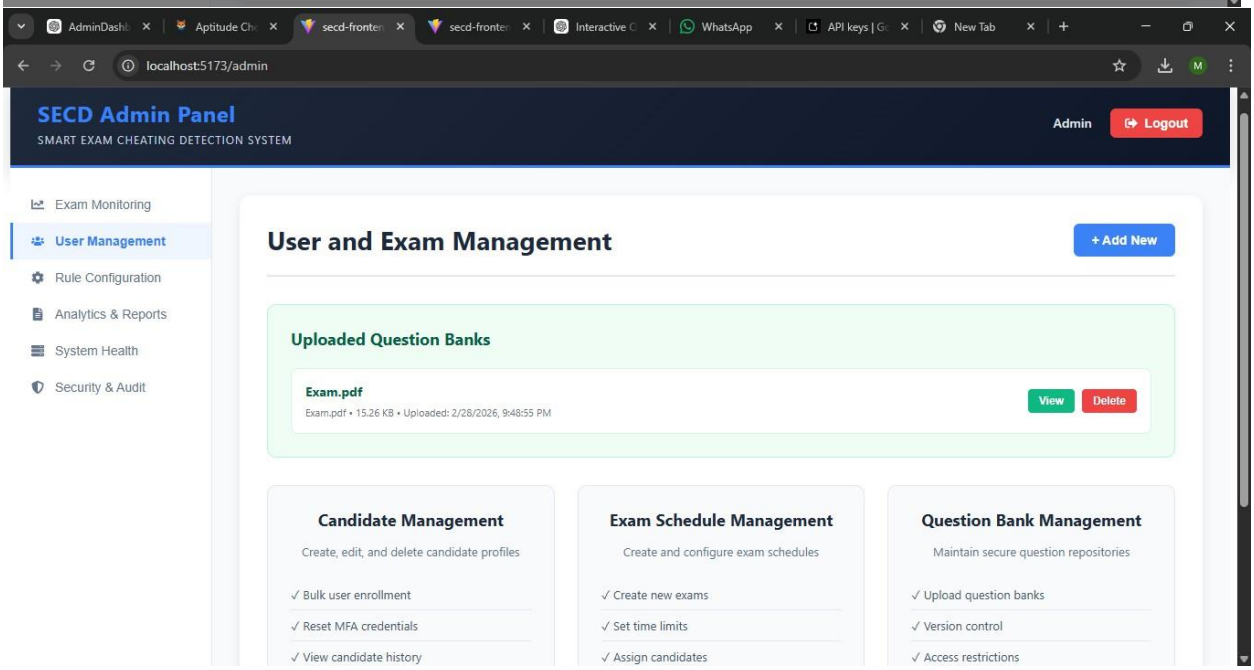
4. Dynamic Question Bank List

```
{uploadedQuestionBanks.map((bank) => (
  <div key={bank.id} class="bank-item">
    <strong>{bank.name}</strong>
    <button onClick={() => viewBank(bank)}>
            View
    </button>
    <button onClick={() => deleteBank(bank.id)}>
            Delete
    </button>
  </div>
))}
```

Output:







Result: Thus, dynamic visual effects such as loading indicators, alerts, conditional exam cards, and question bank lists were successfully implemented in the web application.

Ex No : 11 **Install MySQL and Create Local and Remote Database**

Date : 27/02/26

Aim: To install MySQL, create a local database using MySQL Server, and create a remote database using a cloud database service for the Smart Exam Cheating Detection System

Part 1: Create Local Database using MySQL Server

Procedure (Local Database)

1. Download and install MySQL Community Server from the official MySQL website.
2. Install MySQL Workbench for database management.
3. Open MySQL Workbench or Command Prompt.
4. Connect to the MySQL server using the root user.
5. Create a new database named smart_exam_system.
6. Create required tables such as users, exams, question_banks.
7. Insert sample data and verify the database.

Source Code / Commands (Local Database)

1. Open MySQL

```
mysql -u root -p
```

2. Create Database

```
CREATE DATABASE smart_exam_system;
```

3. Use Database

```
USE smart_exam_system;
```

4. Create Users Table

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  role VARCHAR(20)
);
```

5. Create Exams Table

```
CREATE TABLE exams (
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  title VARCHAR(200),
  total_candidates INT,
  start_time VARCHAR(20),
  status VARCHAR(20)
);
```

6. Create Question Bank Table

```
CREATE TABLE question_banks (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(200),
  file_name VARCHAR(200),
  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

7. Insert Sample Data

```
INSERT INTO users (name, email, role)
VALUES
('Admin','admin@test.com','admin'),
('User','user@test.com','candidate');
INSERT INTO exams (title, total_candidates, start_time, status)
VALUES
('Java Programming Midterm',45,'09:00 AM','ongoing');
```

8. Verify Database

```
SHOW DATABASES;
SHOW TABLES;
SELECT * FROM users;
```

Output (Local Database)

Command

```
SHOW DATABASES;
```

Output

```
+----------------------+
| Database             |
+----------------------+
| exam_system          |
| information_schema   |
| mysql                |
| performance_schema   |
| secd_db              |
| smart_exam_system    |
| sys                  |
+----------------------+
```

Command

```
SELECT * FROM users;
```

Output

```
+- - -+--------+------------------+------------+
| id | name  | email           | role      |
+- - -+--------+------------------+------------+
| 1  | Admin | admin@test.com | admin      |
| 2  | User  | user@test.com  | candidate |
+- - -+--------+------------------+------------+
```

Part 2: Create Remote Database using Cloud MySQL

Procedure (Remote Database)

1. Go to the Railway website (https://railway.app).
2. Create a free account and login.
3. Create a New Project.
4. Click Add Service → MySQL.
5. Railway automatically creates a Cloud MySQL Database.
6. Open the MySQL service → Connect tab.
7. Copy the connection URL and database credentials.
8. Use these details to connect the database with the Flask backend of the Smart Exam Cheating Detection System.

Example MySQL Remote Connection

Connection String Example

mysql://root:password@gondola.proxy.rlwy.net:49890/railway

Example Flask Database Connection

(Backend/app.py)

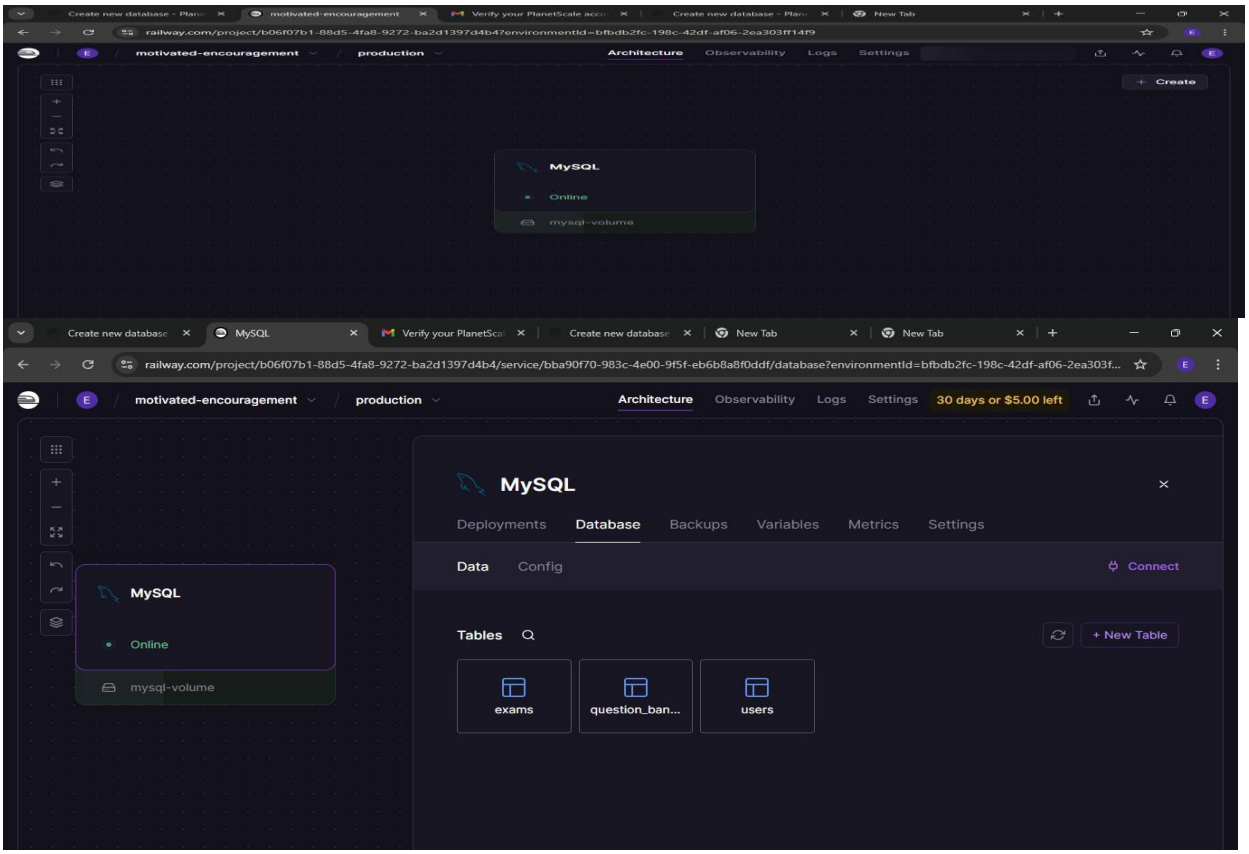
```
1.  from flask import Flask
2.  import mysql.connector
3.  app = Flask(__name__)
4.  db = mysql.connector.connect(
5.  host="gondola.proxy.rlwy.net",
6.  user="root",
7.  password="YOUR_RAILWAY_PASSWORD",
8.  database="railway",
9.  port=49890
10. )
11. print("Remote Database Connected Successfully")
```

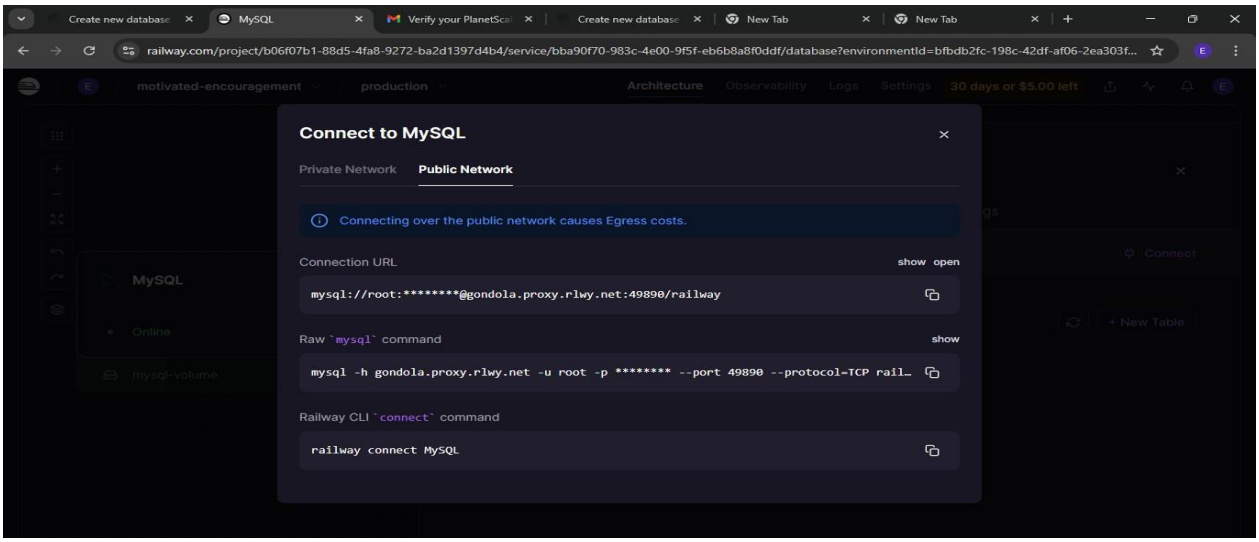
Output (Remote Database)

After successful connection, tables will appear in the cloud database dashboard.

Example Tables

users
exams
question_banks





Result: Thus, MySQL was successfully installed, a local database was created, and a remote cloud database was configured and connected to the Flask backend of the Smart Exam Cheating Detection System.

Ex No : 12 **Create a Full Stack Web Application – Smart Exam Cheating Detection System**

Date : 05/03/26

Aim: To develop a full stack web application for the Smart Exam Cheating Detection System (SECD) that allows administrators to monitor exams, manage users, configure cheating detection rules, analyze reports, and maintain system security using React (Frontend), Flask (Backend), and MySQL (Database).

Source Code / Commands
(Frontend/src/ components /AdminDashboard.jsx)
1. Importing Required Libraries

```
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import {
  FaChartLine,
  FaUsers,
  FaCog,
  FaShieldAlt,
  FaServer,
  FaFileAlt,
  FaSignOutAlt,
  FaBell,
  FaCheckCircle,
  FaExclamationCircle
} from "react-icons/fa";
import QuestionUpload from "./QuestionUpload";
import "../styles/AdminDashboard.css";
```

2. State Variables

```
const [activeTab, setActiveTab] = useState("monitoring");
const [adminData, setAdminData] = useState(null);
const [exams, setExams] = useState([]);
const [alerts, setAlerts] = useState([]);
const [loading, setLoading] = useState(false);
const [showUploadModal, setShowUploadModal] = useState(false);
const [uploadedQuestionBanks, setUploadedQuestionBanks] = useState([]);
```

3. Loading Admin Data

```
useEffect(() => {
  const admin = JSON.parse(localStorage.getItem("admin"));
  setAdminData(admin);
  loadDemoData();
}, []);
```

4. Exam Monitoring Data

```
const loadDemoData = () => {
  setExams([
   {
     id: 1,
     title: "Java Programming Midterm",
     candidates: 45,
     active: 23,
     flagged: 3,
     status: "ongoing",
     startTime: "09:00 AM",
   },  {
       id: 2,
      title: "Database Design Final",
     candidates: 38,
     active: 0,
     flagged: 2,
     status: "completed",
     startTime: "01:00 PM",
    }
 ]);
};
```

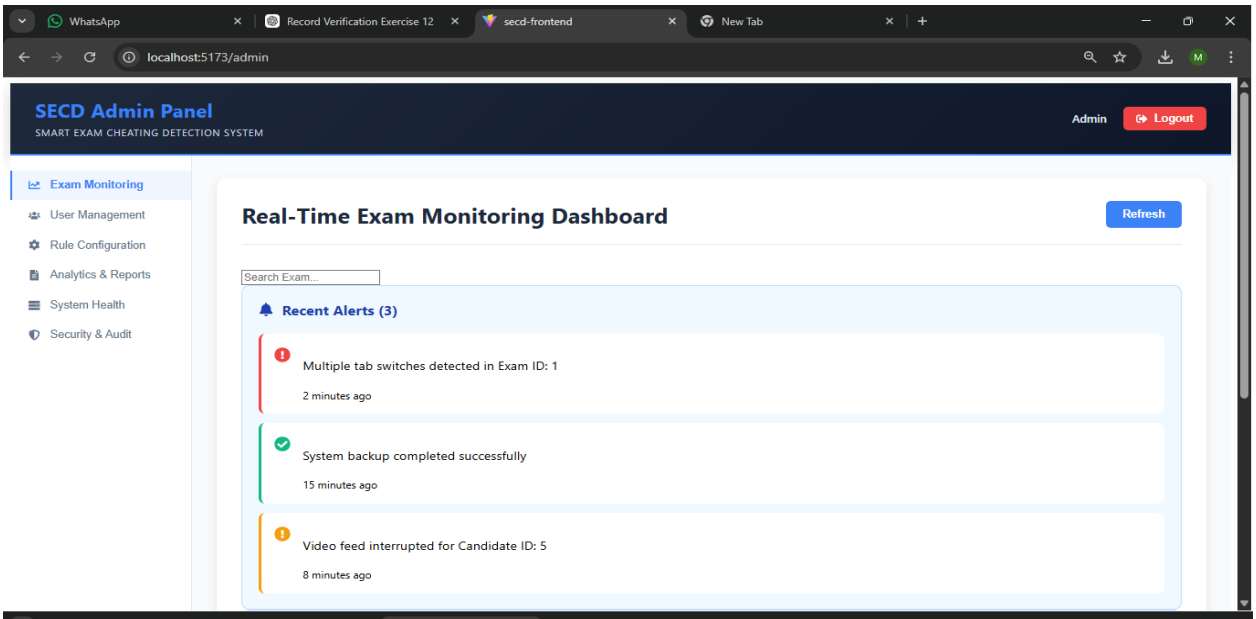
5. Logout Function

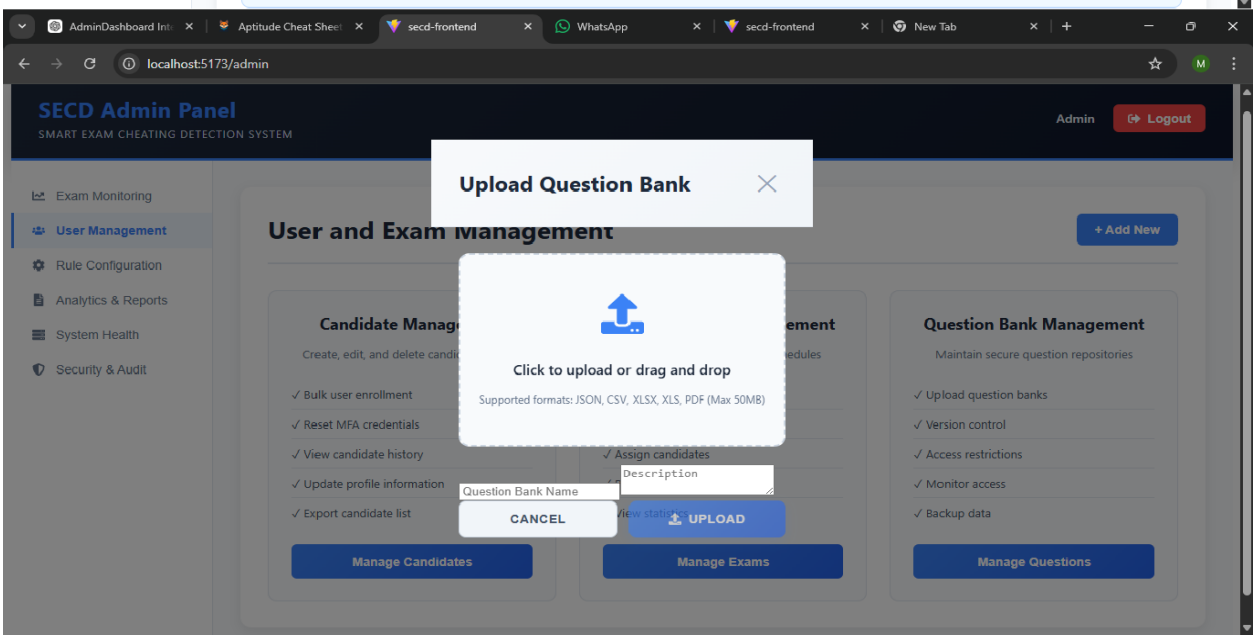
```
const handleLogout = () => {
  localStorage.removeItem("adminToken");
  localStorage.removeItem("admin");
  navigate("/admin-login");
};
```

6. Question Bank Upload

```
const handleQuestionUploadSuccess = (questionBankData) => {
    setUploadedQuestionBanks([
    {
      id: Date.now(),
..............questionBankData,
      uploadedAt: new Date().toLocaleString(),
     },
...........uploadedQuestionBanks,
  ]);
  setShowUploadModal(false);
};
```

Output:





Result: The Smart Exam Cheating Detection System Admin Dashboard was successfully developed using React for the frontend, Flask for backend services, and MySQL for database management.